



UNIVERZITET U BEOGRADU

ELEKTROTEHNIČKI FAKULTET

Katedra za elektroniku i digitalne sisteme

SISTEMI U REALNOM VREMENU

Projektni izveštaj g53

Student	Vasilije Čarapić
Broj indeksa	2022/0218
Grupa	$(53)_{10} = 110101_2$
AB	11 → Kanali A0, A1 Akvizicija na 1500 ms
C	0 → Softverski tajmer
D	1 → Binarni semafor
E	0 → Mailbox-ovi (Queue dužine 1) sa overwrite-om
F	1 → Uključi diodu ako je vrednost MANJA od granične

1. Postavka zadatka

Implementirati sistem zasnovan na FreeRTOS-u za akviziciju analognih signala sa MSP430, obradu i signalizaciju putem UART-a i LED dioda.

Konfiguracioni parametri

Na osnovu grupe $(53)_{10} = 110101_2$ definisani su sledeći parametri:

- $AB = 11$ — Akvizicija sa kanala A0 i A1 na svakih 1500 ms
- $C = 0$ — Pokretanje akvizicije softverskim tajmerom
- $D = 1$ — Obaveštavanje xTask1 putem binarnog semafora
- $E = 0$ — Upis srednje vrednosti u mailbox-ove (Queue dužine 1, overwrite)
- $F = 1$ — Dioda se pali ako je srednja vrednost MANJA od granične

Opis zahteva

Potrebno je implementirati odloženu obradu prekida (deferred interrupt processing) AD konvertora:

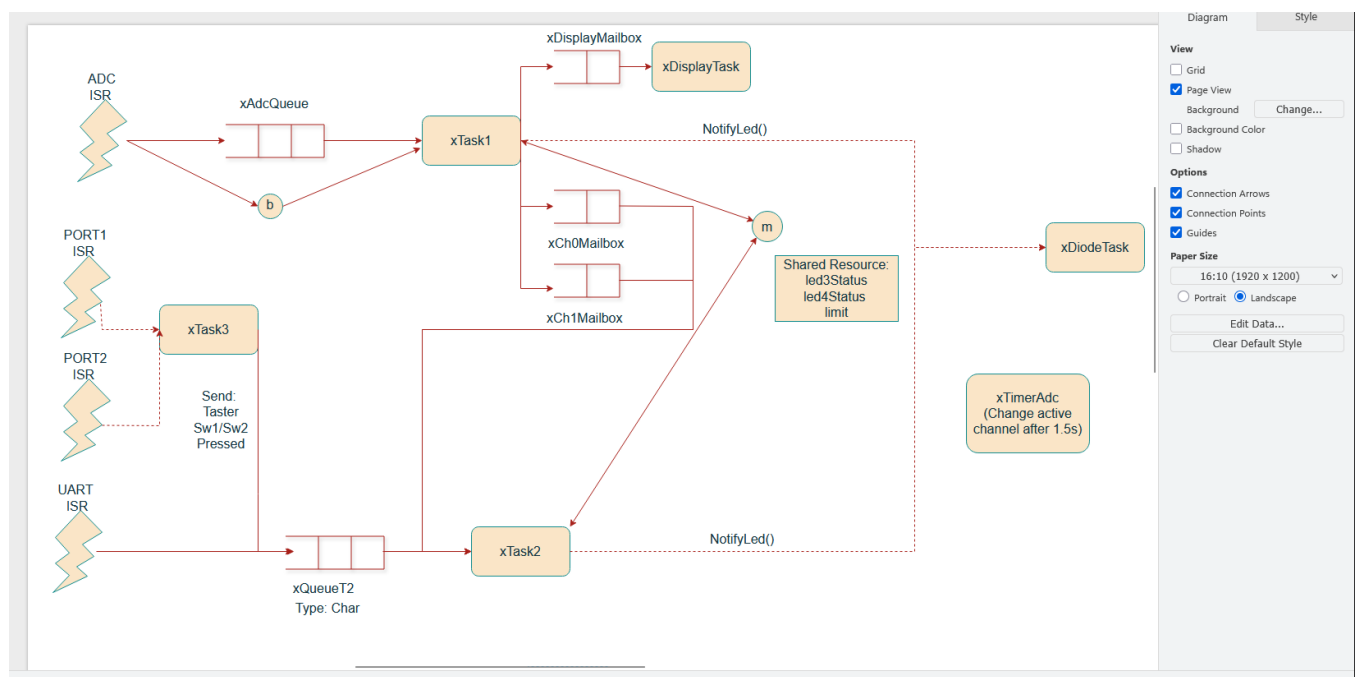
- U prekidnoj rutini ADC upisuje se rezultat konverzije u red sa porukama (xAdcQueue)
- xTask1 se obaveštava o pristizanju poruke putem binarnog semafora
- Poruka sadrži informaciju o kanalu i gornjih 12 bita rezultata AD konverzije
- xTask1 računa srednju vrednost poslednjih 4 uzorka po kanalu i upisuje u mailbox-ove

Preko UART-a moguće je zadati graničnu vrednost za signalizaciju. Task xTask2 obrađuje primljene vrednosti i pritisnut taster (S1/S2), inkrementira/dekrementira granicu i šalje status dioda.

2. Arhitektura sistema

Arhitektura sistema prikazana je na sledećoj slici. Sistem koristi sledeće FreeRTOS elemente:

- Redovi sa porukama (Queue)
- Taskovi (Tasks)
- Prekidne rutine (ISR)
- Binarni semafore
- Mutex semafore
- Timer



Slika 1 — Arhitektura sistema (dijagram toka i međuzavisnosti taskova)

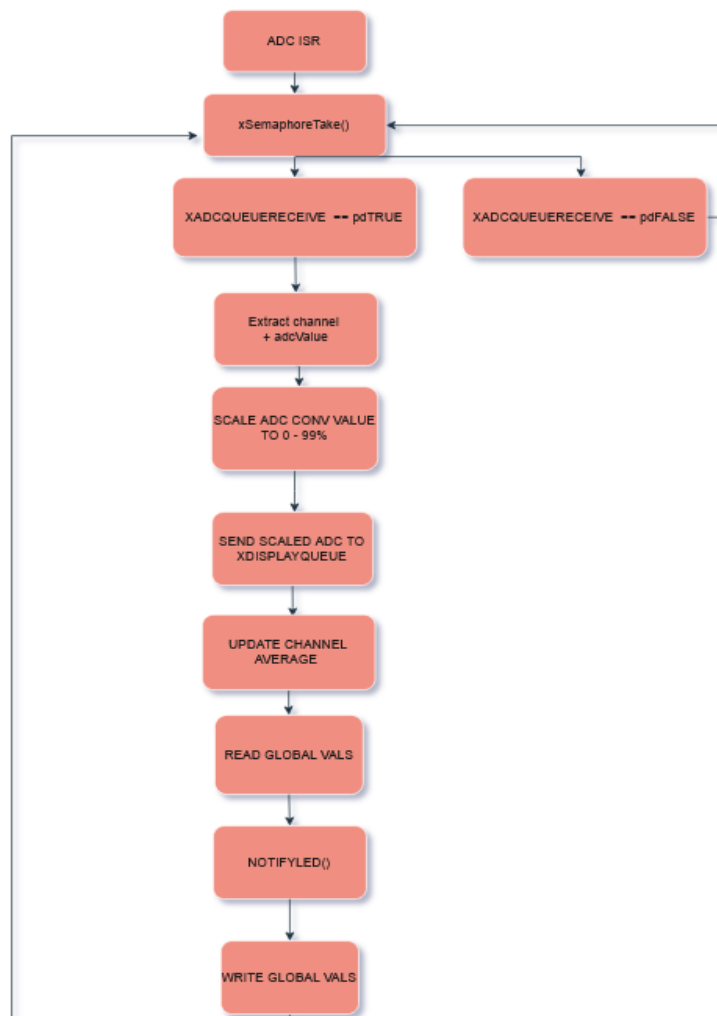
3. Opis rada taskova

3.1 xTask1 — ADC obrada i srednja vrednost

xTask1 je blokiran na redu sa porukama xAdcQueue. Kada se red deblokira, binarni semafor xTask1 obaveštava task da je novi podatak dostupan, nakon čega task preuzima podatak iz reda.

Tok obrade unutar xTask1:

- Izvlači podatke o kanalu i vrednosti AD konverzije
- Skalira vrednost potencijometra na opseg 0%–99%
- Upisuje skaliranu vrednost u xDisplayMailbox za prikaz na displeju
- Poziva processAndSend() koja računa srednju vrednost poslednjih 4 uzorka i upisuje u xCh0Mailbox ili xCh1Mailbox
- Uzima mutex, čita globalne promenljive, smešta u lokalne, poziva NotifyLed()
- NotifyLed() proverava da li je srednja vrednost < limit i notifikuje xDiodeTask
- Po povratku iz NotifyLed() ažurira globalne promenljive led3Status i led4Status



3.2 xTask2 — Obrada UART-a i tastera

xTask2 je najsloženiji task u sistemu sa dvojakom ulogom: obrađuje granične vrednosti primljene putem UART-a i statusne poruke od pritisnutih tastera.

Task je blokiran na xQueueT2 (tip char, prima karakter po karakter). Definisan je enum tip za identifikaciju akcije:

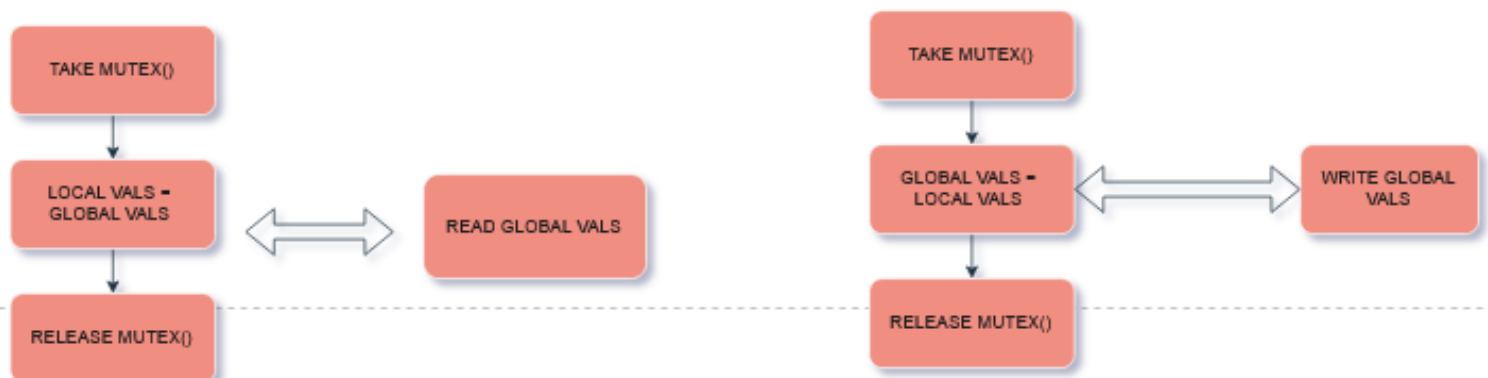
```
typedef enum { ACTION_UART_VAL_SENT, ACTION_SW_PRESSED, ACTION_UNDEFINED } action_t;
```

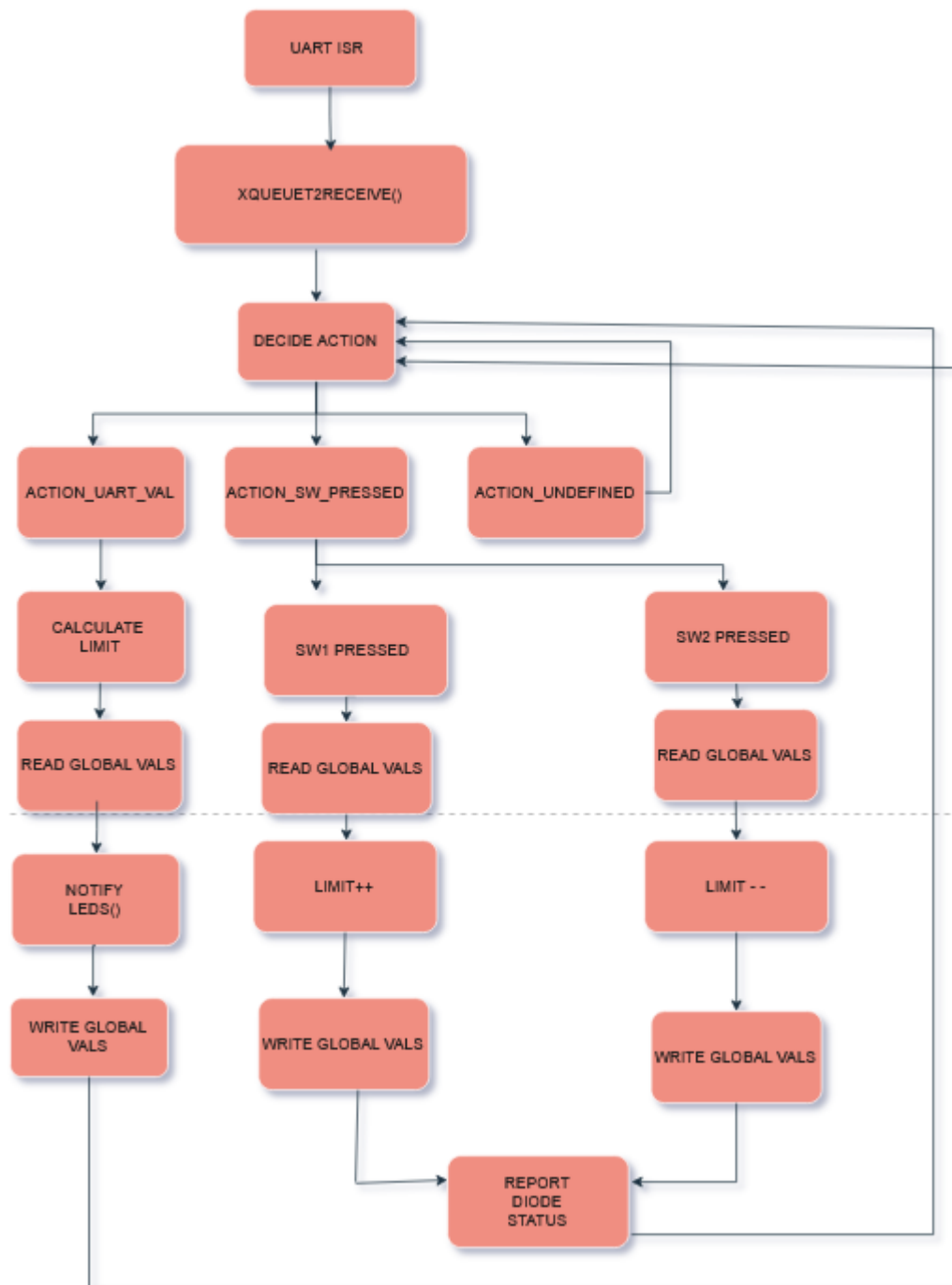
- **ACTION_UART_VAL_SENT** — primljena granična vrednost putem UART-a
- **ACTION_SW_PRESSED** — pritisnut taster S1 ili S2
- **ACTION_UNDEFINED** — nastavak parsiranja dok podatak nije kompletan

Obrada ACTION_UART_VAL_SENT: konverzija ASCII → decimalni format, poziv notifyLed() za ažuriranje stanja dioda uz zaštitu mutex-om.

Obrada ACTION_SW_PRESSED: poziv updateLimit() za promenu granične vrednosti, pa reportDiodeStatus() za UART izveštaj o stanju dioda.

Radi lakseg prikazivanja uvešćemo pomoćne grafike koji će označavati uzimanje mutex-a i pisanje / čitanje iz globalnih promenljivih:



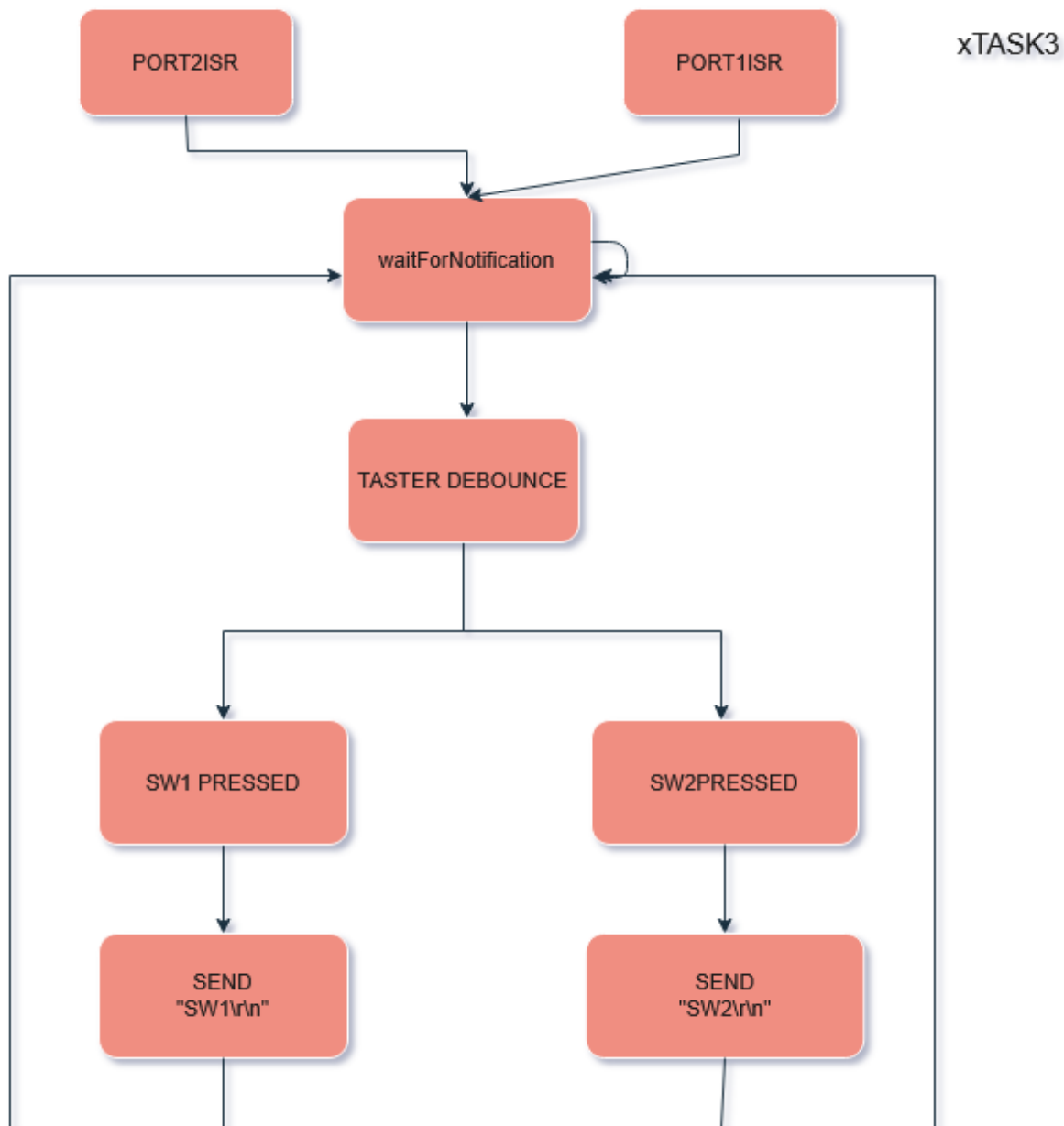


3.3 xTask3 — Praćenje tastera

xTask3 je klasičan buttonTask koji prima notifikacije od buttonISR-a. Pošto su tasteri na različitim portovima, koriste se dve prekidne rutine:

- **PORT1 Vector ISR** — za taster S1
- **PORT2 Vector ISR** — za taster S2

Nakon detekcije pritiska tastera, task šalje identifikator pritisnutog tastera u xQueueT2 na dalju obradu od strane xTask2.

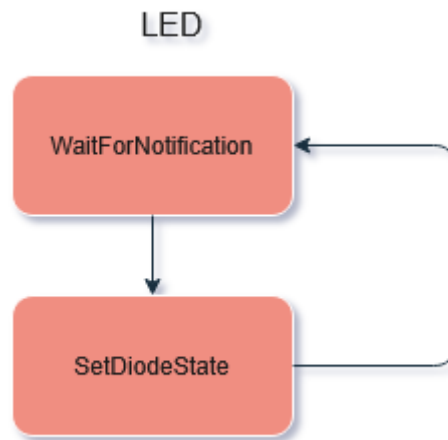


3.4 xDiodeTask — Upravljanje LED diodama

xDiodeTask je kompaktan task koji prima notifikacije od xTask1 ili xTask2. Logika rada:

- Čeka notifikaciju (blokiran)
- Po prijemu notifikacije, čisti staro stanje dioda
- Ako je notifikacija za isključivanje — diode su već ugašene, prelazi na sledeću iteraciju
- Ako je notifikacija za paljenje — pali odgovarajuću diodu

Vizuelni prikaz toka xDiodeTask-a:

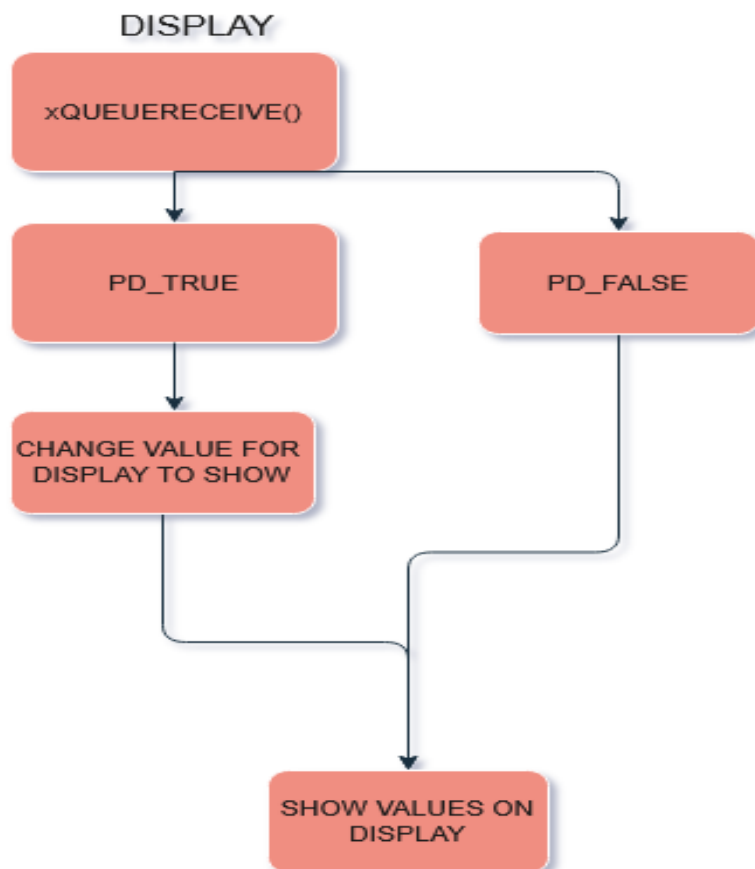


Slika 2 — Dijagram toka xDiodeTask

3.5 xDisplayTask — prikaz displeja

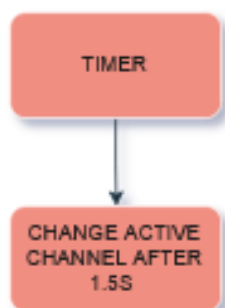
xDisplayTask kontinualno prikazuje vrednosti na oba 7-segmentna displeja korišćenjem multipleksiranja:

- Prikazuje vrednost 5ms na prvom displeju, pa 5ms na drugom
- Koristi vTaskDelay za tačan period od 5ms
- Ljudsko oko ne može da detektuje treperenje ispod 20ms — 10ms je dovoljno za iluziju stalnog prikaza
- Konstantno proverava xDisplayQueue od strane xTask1: ako je stigao novi podatak, ažurira prikazanu vrednost; u suprotnom prikazuje staru



3.6 Softverski timer xAdcTimer

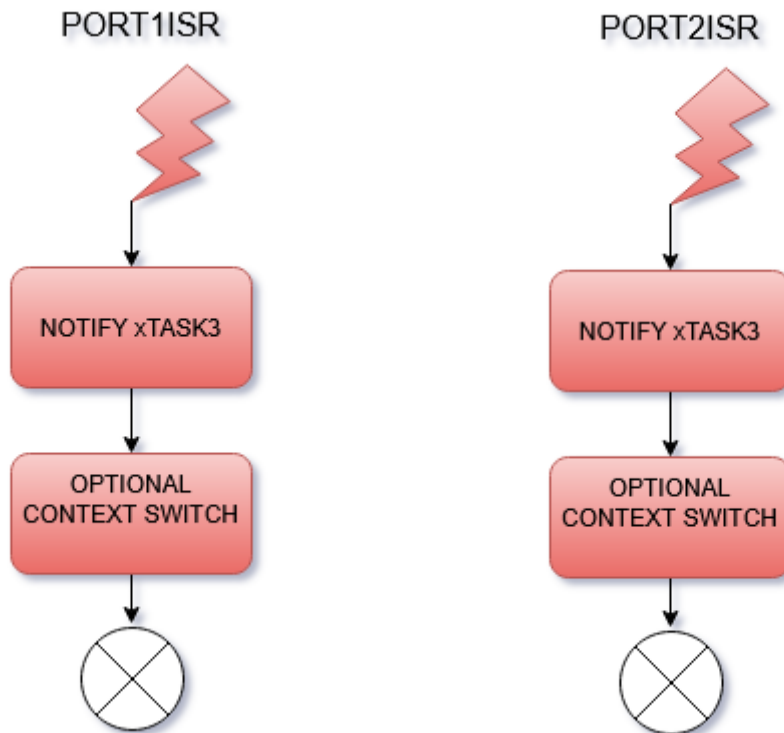
Menja aktivan kanal na svakih 1500ms:



3.7 Prekidne rutine

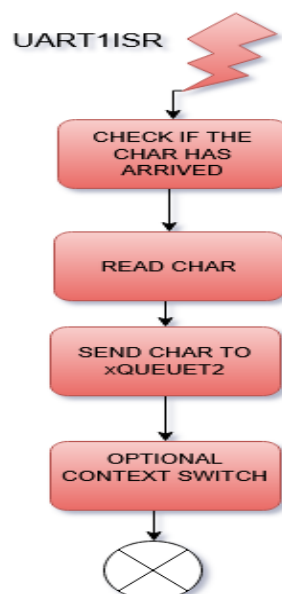
vPORT1ISR i **vPORT2ISR** (Spoljašnji prekidi)

Ove rutine reaguju na pritisak tastera S1 (Port 1) i S2 (Port 2). One vrše prosleđivanje notifikacije tasku 3 preko funkcije **vTaskNotifyGiveFromISR()**.



vUARTISR

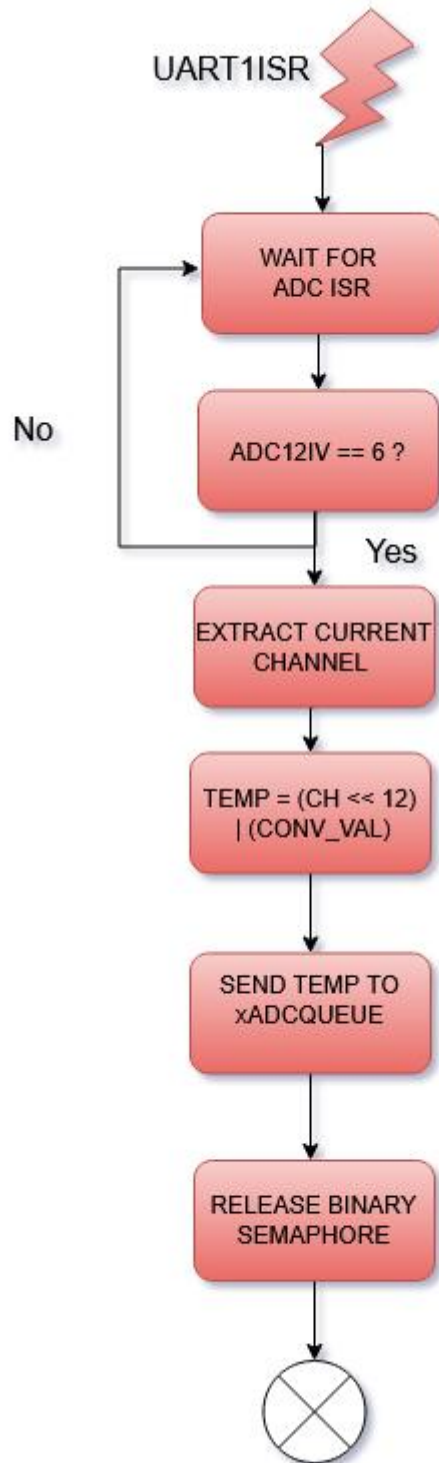
Aktivira se kada periferija uspešno primi bajt sa serijske veze. Rutina trenutno čita karakter iz registra **UCA1RXBUF** (čime se prazni hardverski bafer) prosleđuje ga na kraj reda **xQueueT2** kako bi ga **xTask2** kasnije obradio.



vADC12ISR (Prekid završetka ADC konverzije)

Pokreće se kada se završi ADC konverzija određenog kanala.

Prekidna rutina uzima 12-bitnu vrednost iz registra ADC12MEM0, dodaje joj informaciju o aktivnom kanalu na višim bitovima, te podatak šalje u red **xAdcQueue** uz istovremeno davanje binarnog semafora **xAdcBinarySem** koji odblokira xTask1.



3.8 Mehanizam buđenja taska višeg prioriteta

U svim prikazanim prekidnim rutinama (ISR) koristi se specifičan FreeRTOS mehanizam za očuvanje osobina sistema u realnom vremenu.

Promenljiva **xHigherPriorityTaskWoken** se inicijalizuje na vrednost **pdFALSE**. Kada se iz ISR-a pozove FreeRTOS API funkcija **FromISR** (poput slanja u red, davanja semafora ili slanja notifikacije), sam kernel proverava da li je taj događaj odblokirao task koji ima veći prioritet od taska koji je trenutno bio prekinut. Ako jeste, kernel postavlja ovu promenljivu na **pdTRUE**.

Makro **portYIELD_FROM_ISR(xHigherPriorityTaskWoken)** se poziva na samom kraju svake prekidne rutine. Ukoliko je promenljiva postavljena na **pdTRUE**, ovaj makro zahteva trenutnu promenu konteksta pre izlaska iz prekida. To znači da se procesor, umesto da se vrati u prekinuti task nižeg prioriteta, odmah prebacuje na izvršavanje odblokiranog taska višeg prioriteta. Time se minimizuje kašnjenje sistema.

4. Zaključak

Sistem koristi binarni semafor za komunikaciju **ADC ISR → xTask1**, mailbox-ove za distribuciju srednjih vrednosti, i red **xQueueT2** za obradu UART i taster događaja u okviru **xTask2**. Mutex štiti globalne promenljive od race condition-a između taskova. **xDisplayTask** obezbeđuje stabilan vizuelni prikaz bez problema kao što je treperenje displeja, multipleksiranjem displeja na 10ms periodi.